Lecture 1.3 Variables, Expressions and Operators.

To start class: Open your notebook for this lecture

Lecture Homepage -> scroll down to Week 1, Kinsella &
Goodfellow Link -> Lecture 1.3 -> “Jupyter Notebook file”

Introducing Python

Programs are stored in .py files , and ran with commands in the terminal such as: ‘python tmp.py’

Python	**Many other languages**
Interpreted	Compiled
Run line by line	Compile first
No variable declarations	Declarations required
Variable types can change	Types are fixed

Python runs code line by line, so you can use it like a calculator, and variables don’t need types or declarations ahead of time, which is what we will go through today

Introducing Python cont'd

No end-of-instruction separators, such as semicolons (like in C or Java)

in c:

```
>> int x = 5;
>> int y = 10;
```

Python uses line breaks (new lines) instead:

- In Python:

```
>> x = 5
>> y = 10
```

- In Python, you can also do something like

```
>> x = 5; y = 10
```

But again, Python knows an instruction ends when the line ends

Introducing Python cont'd

Comment out any line (the code will ignore it when running) with a '#' at the start of the line

Whitespace matters: exactly 4 spaces means indentation, Indented lines form a block of code)

```
>> if x > 0:
>>     print("Positive")
>>     print("Done")
```

Note the indent to run this block of code!

Everything in Python is treated as an object (with a type and behavior)

Variable Types

- A **type** is a set of values and the operations that can be performed on those values.

- Every value in Python has a *type*: e.g., number, text, true/false,

Data Type	Description	Example
`int`	Whole numbers	`3, -10, 42`
`float`	Decimal numbers	`3.14, -0.5`
`str`	Text (strings)	`"hello", "123"`
`bool`	True/False values	`True, False`

- "3" isn't the same as 3

- Is important because it partly determines allowed operations

- Python figures out the type automatically

Everything in Python is treated as an object (with a type and behavior)

A quick word about checking what the type of value is:
Python gives us a built-in function that tells us the type of a value

```
>> type(5)
>> type(3.14)
>> type("hello")
>> type(True)
```

… quick example in notebook about different types of *numerical* objects…

Note: functions in python all have func(), e.g.,

```
>> print()
>> type()
>> len()
>> int()
>> exit()
```

We'll work with these today!

*When you see a name followed by parentheses,
Python is doing something with the value inside*

Working with numbers: Python can evaluate expressions exactly like a calculator (but with many more libraries available)

Operator	Operation	Expression	English description	Result
+	addition	11 + 56	11 plus 56	67
-	subtraction	23 - 52	23 minus 52	-29
*	multiplication	4 * 5	4 multiplied by 5	20
**	exponentiation	2 ** 5	2 to the power of 5	32
/	division	9 / 2	9 divided by 2	4.5
//	integer division	9 // 2	9 divided by 2	4
%	modulo (remainder)	9 % 2	9 mod 2	1

... quick example in notebook about what these operations do for different types of numerical objects vs. strings...

Type: str (pronounced string)

- *str:* string is a sequence of characters

- Start and end with single quotes (') or double quotes (")

ex. 'hello', "What is 10 * (2 + 9)?"

1. Just like writing in English, the quote type must match (i.e. 2 singles or 2 doubles, not 1 of each)

… what happens if we apply arithmetic operations to strings…

ex. "hello" + " " + "Tom"

Arithmetic Operator Precedence

When multiple operators are combined in a single expression, the operations are evaluated in order of precedence (from left to right)

Add something here

Variables, Assignment, and Memory in Python

The most basic/widely done operation in any computer program is to assign a value to a variable.

Assignment statements

Variable_name = value

The right-hand side is evaluated first, then point the value in the memory using the variable on the left

```
>>> x = 10
```

Variables do not “store” values really; variables point to objects in memory.

- 10 is an object in memory
- x is a variable name (label)
- x points to the object 10

Memory Visualization Example

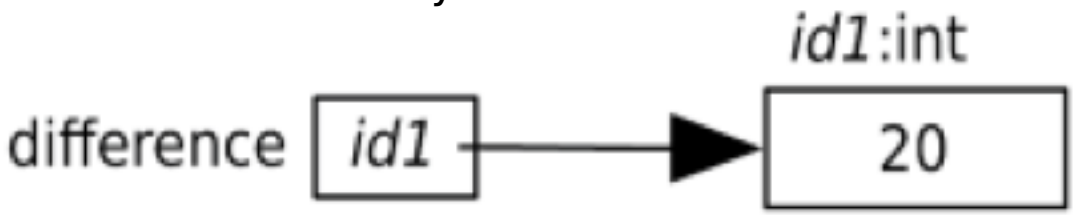
```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

1. Evaluate the expression on the right of the = sign -> 20. This produces the value 20, which we'll put at memory address id1.

2. Make the variable on the left of the = sign, difference, refer to 20 by storing id1 in difference.

3. id1 This distinguishes one object from another.

`20` is stored at memory location `id1`



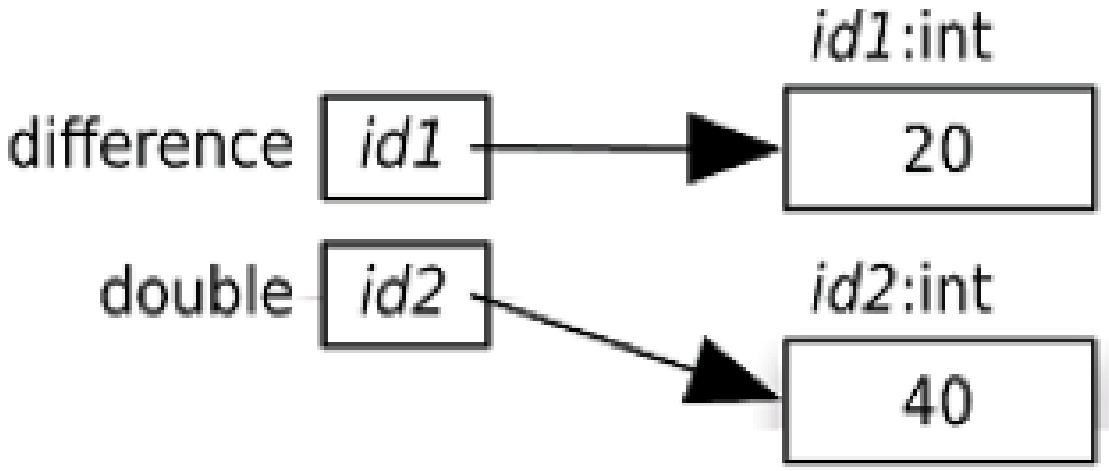
Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

1.Evaluate the expression on the right of the = sign: `2 * difference`. As we see in the memory model, difference refers to the value `20`, so this expression is equivalent to `2*20`, which produces `40`

2. `40` is stored at memory location `id2`

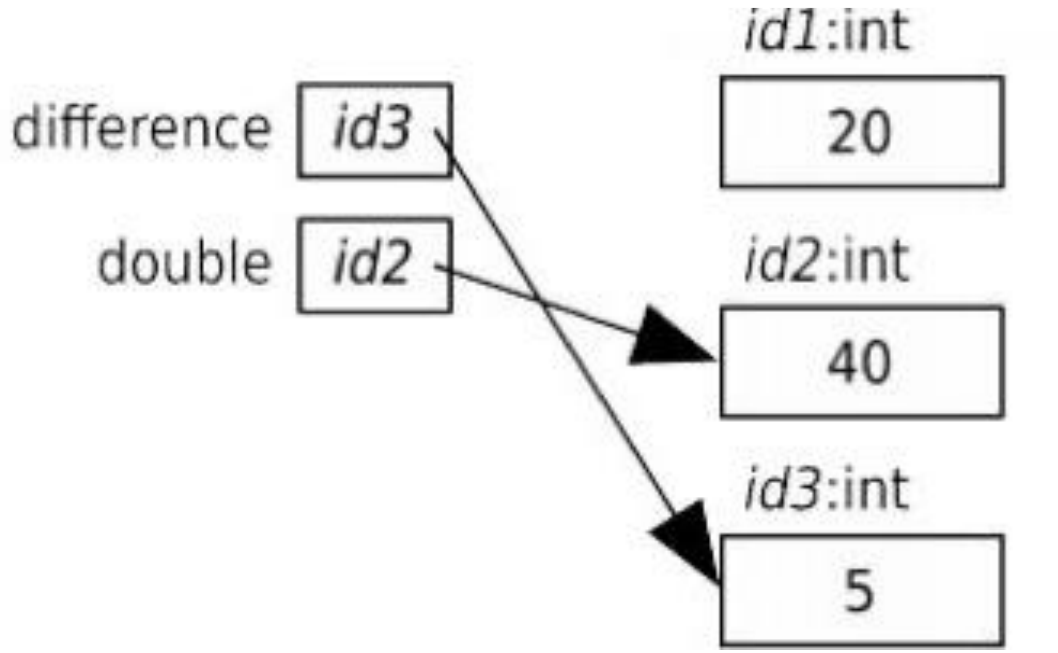
3. And the variable on the left of the = sign, `double`, refer to `40` by storing `id2`



Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

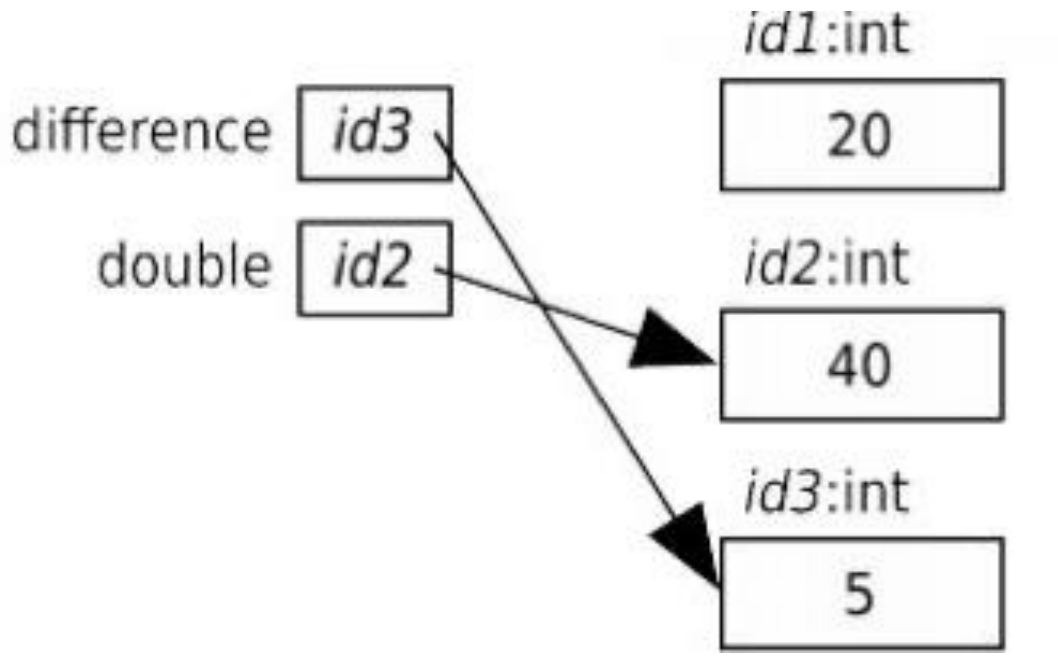
1. Evaluate the expression on the right of the = sign: `5`. This produces the value `5`, which we'll put at the memory address `id3`.
2. Make the variable on the left of the = sign, difference, refer to `5` by storing `id4` in difference.



Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

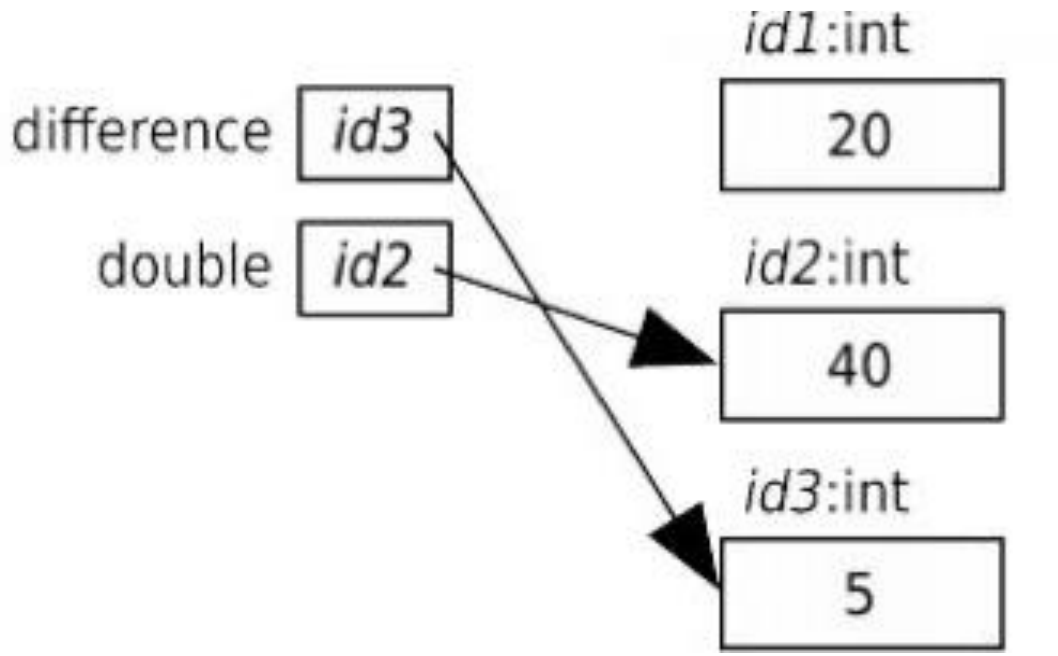
1. Variable double still contains `id2`, so it still refers to `40`. Neither variable refers to `20`
2. Memory for `id1` is cleared out typically (c**an be an issue in ipynb!!**)



Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

1. Variable double still contains `id2`, so it still refers to `40`. Neither variable refers to `20`
2. Memory for `id1` is cleared out typically (c**an be an issue in ipynb!!**)



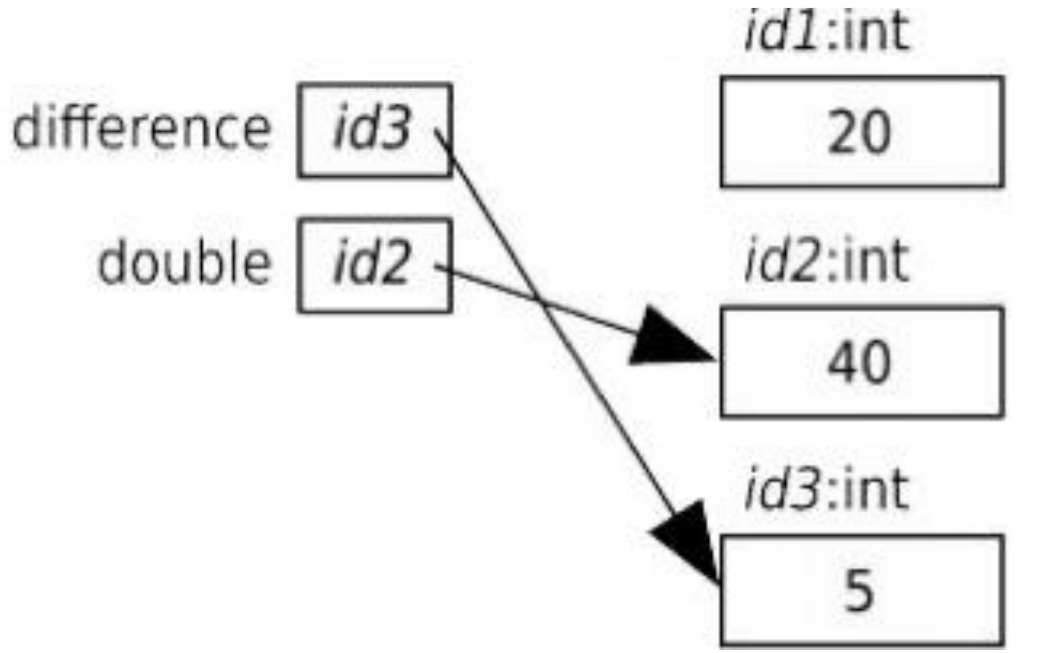
What would happen if we ran simply:

```
>>> difference
```

Memory Visualization Example

```
>>> difference = 20
>>> double = 2 * difference
>>> double
40

>>> difference = 5
>>> double
40
```

1. Variable double still contains `id2`, so it still refers to `40`. Neither variable refers to `20`
2. Memory for `id1` is cleared out typically (**can be an issue in ipynb!!**)



What would happen if we ran simply:

```
>>> difference + double
```

Variables and Memory Practice

Let's go experiment with variable creation and assignments

- Assignment statements
- Declaring variables
- Different variable names
- Memory locations
- Use `print` to keep track / debug

Open your notebook

Click Link:
2. Variables and Memory

Variables Types

Now that we have some experience with variable assignment and integers, Let's go experiment with some strings

- "int"s vs "float"s
- The "str" type
- Using types and variables in expressions
- Combining our type knowledge with some arithmetic operations

Open your notebook

Click Link:
3. Different Types of Variables

Augmented Assignment Operations

Operator	Expression	Identical Expression	English description
+=	x=7 x += 2	x=7 x=x+2	x refers to 9
-=	x=7 x -= 2	x=7 x=x-2	x refers to 5
*=	x=7 x *= 2	x=7 x=x*2	x refers to 14
/=	x=7 x /= 2	x=7 x=x/2	x refers to 3.5
//=	x=7 x //= 2	x=7 x = x // 2	x refers to 3
%=	x=7 x %= 2	x=7 x=x%2	x refers to 1
**=	x=7 x **= 2	x=7 x = x ** 2	x refers to 49

Open your notebook

Click Link:

4. Augmented Assignment Operators